

**SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU
FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I
INFORMACIJSKIH TEHNOLOGIJA OSIJEK**

Sveučilišni diplomski studij Računarstvo

Umjetna inteligencija i robotika

**Klasifikacija vrsta gljiva iz terenskih snimki pomoću
vizualnog transformera s LoRA adapterima**

Seminarski rad

Katarina Rukavina

Osijek, 2026.

SADRŽAJ

1. UVOD.....	1
2. PREGLED PODRUČJA I PROBLEMATIKE.....	2
2.1. Vizualni transformer (ViT).....	2
2.2. LoRA.....	3
2.3. Metrike za evaluaciju modela.....	4
3. SKUP PODATAKA.....	6
3.1. Izvor i sastav skupa podataka.....	6
3.2. Podjela skupa podataka.....	7
3.3. Predobrada slika i augmentacije podataka.....	7
4. METODOLOGIJA.....	9
4.1. Predtrenirani model.....	9
4.2. LoRA i puni fine tuning.....	9
4.3. Postupak treniranja.....	10
4.4. Ablacijski eksperimenti.....	11
4.5. Mjerenje memorije i računalne učinkovitosti.....	12
4.6. Evaluacijski postupak.....	12
5. REZULTATI.....	14
5.1. Usporedba LoRA i full fine tuninga.....	14
5.2. Primjeri pogrešne klasifikacije.....	18
5.3. Rezultati ablacijskih eksperimenata.....	18
5.4. Preporuke kod treniranja manjih skupova podataka.....	21
6. ZAKLJUČAK.....	22
LITERATURA.....	23
POPIS SLIKA.....	24

1. UVOD

Klasifikacija na temelju terenskih fotografija često je izazovna jer fotografije nisu slikane u kontroliranim uvjetima. Svaka slika je slikana pod različitim osvjetljenjem, iz različitog kuta te često sadrži i dosta šuma. Ovakve skupove podataka karakterizira nedovoljan broj podatkovnih primjera i nejednaka raspodjela po klasama.

Zadatak ovog rada je istrenirati vizualni transformer za klasifikaciju vrsta gljiva na temelju terenskih fotografija, uz primjenu LoRA adaptera, te usporediti dobivene rezultate s pristupom punog fine-tuninga na istom predtreniranom backboneu, uz identične augmentacije podataka. Dobiveni modeli vrednuju se uporabom različitih metrika evaluacije. Dodatno, rad ispituje utjecaj hiperparametara LoRA metode na rezultate i memoriju tijekom treniranja. Iz provedenih pokusa izvode se preporuke za treniranje modela na malim skupovima podataka.

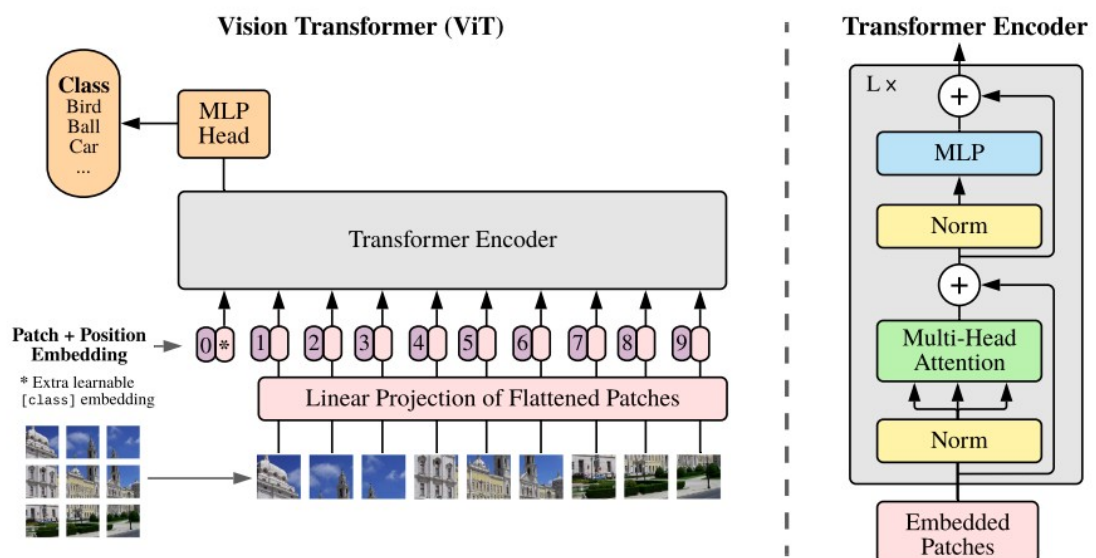
U drugom poglavlju opisane su teorijske osnove korištenih metoda : arhitektura vizualnog transformera, princip rada LoRA adaptera te metrike korištene za evaluaciju. Treće poglavlje opisuje skup podataka terenskih fotografija gljiva, postupak podjele na skupove za treniranje, validaciju i testiranje, te primijenjene augmentacije. U četvrtom poglavlju opisana je struktura modela, postupak treniranja LoRA i punog fine-tuning modela te ablacijski eksperimenti nad rangom i ciljnim slojevima LoRA adaptera. Peto poglavlje donosi rezultate eksperimenata i njihovu interpretaciju. Rad završava zaključkom, u kojem su sažeti glavni doprinosi i nalazi te predložena moguća proširenja budućeg istraživanja.

2. PREGLED PODRUČJA I PROBLEMATIKE

2.1. Vizualni transformer (ViT)

Transformer arhitekture [1] su one čiji se pristup sastoji od predtreniranja na velikom skupu podataka, nakon čega slijedi *fine-tuning* na manjem skupu podataka specifičnom za određeni zadatak. Računalna učinkovitost i skalabilnost transformera omogućile su treniranje modela enormnih veličina.

Dok konvolucijske mreže obrađuju sliku kao cjelovitu rešetku piksela, vizualni transformeri slike dijele u slijed spljoštenih 2D zakrpa. Svaka zakrpa tretira se kao token kod obrade prirodnog jezika. Ukupan broj dobivenih zakrpa ujedno predstavlja i efektivnu duljinu ulaznog slijeda za transformer. Pomoću postupka trenirajuće linearne projekcije spljoštene 2D zakrpe preslikavaju se u prostor dimenzije. Kao izlaz iz jednadžbe dobije se slijed *patch embeddingsa*. Na početak slijeda dodaje se *learnable embedding* čije stanje na izlazu transformer enkodera služi kao reprezentacija slike. Za razliku od kovolucije koja je po prirodi prostorno organizirana, transformeri nemaju urođeni osjećaj za prostorne odnose. Kako bi se zadržala informacija o položaju zakrpa, njihovim se *embeddingsima* dodaju *position embeddings*. Dobiveni slijed vektora predstavlja ulaz u enkoder koji je sastavljen od L uzastopnih slojeva. Svaki sloj naizmjenično primjenjuje MSA i MLP blokove. Prije svakog bloka, sloj se normalizira i nakon svakog bloka postoje rezidualne veze. Primjer arhitekture vizualnog transformera prikazan je na slici 2.1.



Slika 2.1 Izvor: Dosovitskiy i sur., 2021.

Postoji mehanizam koji za svaki element ulaznog niza računa tri vektora : *query*, *key* i *value*. Oni se računaju gore spomenutom linearnom projekcijom ulaza. Težine pažnje između svaka dva elementa niza izračunavaju se kao skalarni umnožak njihovih *query* i *key* vektora, nakon čega slijedi softmax normalizacija. Te težine zatim određuju koliko će svaki element niza pridonijeti reprezentaciji svakog drugog elementa kroz ponderiranu sumu *value* vektora. MSA je varijanta ovog mehanizma koja paralelno pokreće više ovakvih operacija s odvojenim naučenim projekcijama, čime model istovremeno može hvatati različite vrste odnosa između zakrpa slike.

Vizualni transformer ima znatno manje ugrađene pretpostavke specifične za slike (induktivnu pristranost) u odnosu na konvolucijske mreže. Kod CNN-a, lokalnost (svaki filter gleda samo malu okolinu piksela), dvodimenzionalna susjedska struktura i translacijska ekvivarijantnost (isti uzorak prepoznaje se bez obzira gdje se nalazi na slici) ugrađeni su u samu arhitekturu, u svaki sloj mreže. Kod ViT-a, jedino su MLP slojevi lokalni i translacijski ekvivarijantni, dok su MSA slojevi globalni što znači da svaka zakrpa od početka gleda sve ostale zakrpe na slici, bez ugrađene pretpostavke o susjedstvu. Dvodimenzionalna prostorna struktura koristi se vrlo rijetko: na samom početku, kod razdiobe slike na zakrpe, te kod prilagodbe pozicijskih ugrađivanja pri promjeni rezolucije slike. Osim za navedene situacije model mora od nule naučiti sve prostorne odnose između dijelova slike koje bi CNN imao već ugrađene arhitekturom. Zbog toga vizualni transformer često ima više parametara nego CNN, jer uči i prostorne odnose. Potrebno mu je više podataka za učenje nego CNN-u.

2.2. LoRA

Veliki predtrenirani modeli [2] često se prilagođavaju različitim zadacima pomoću *fine-tuninga*, pri čemu se ažuriraju svi parametri modela. Model se inicijalizira predtreniranim težinama, a zatim se svi njegovi parametri ažuriraju tijekom treniranja na novom zadatku, praćenjem gradijenta funkcije gubitka. Glavni nedostatak takvog pristupa je što za svaki novi zadatak treba pohraniti kopiju velikog modela identične veličine. Ovo kod velikih modela postaje praktički neizvedivo u pogledu memorije i računalnih resursa potrebnih za treniranje. Kako bi se smanjili memorijski i računalni zahtjevi, razvijene su metode koje treniraju samo dio parametara ili dodaju male vanjske module.

Jedna od takvih metoda je LoRA koja se temelji na pretpostavci da se promjene težina potrebne za prilagodbu modela određenom zadatku mogu predstaviti matricama niskog ranga. LoRA zamrzava originalne težine modela i uči samo male matrice niskog ranga koje predstavljaju promjenu težina. Prednosti ove metode su manji memorijski zahtjevi, jednostavno prebacivanje između zadataka te učinkovitije treniranje.

Umjesto da se za svaki gusto povezani sloj modela uči puna matrica promjene ΔW jednake dimenzije kao originalna matrica težina, LoRA tu promjenu ograničava tako da je predstavlja umnoškom dviju znatno manjih matrica:

$$W_0 + \Delta W = W_0 + BA$$

gdje je $W_0 \in \mathbb{R}_{d \times k}$ originalna, zamrznuta predtrenirana matrica težina, $B \in \mathbb{R}_{d \times r}$ i $A \in \mathbb{R}_{r \times k}$ su dvije nove, trenabilne matrice, a r je rang to jest hiperparametar koji je znatno manji od $\min(d, k)$.

Tijekom treniranja, originalna matrica W_0 ostaje potpuno zamrznuta i ne prima gradijentna ažuriranja. Jedini parametri koji se uče jesu elementi matrica A i B . *Forward pass* za ulazni vektor x tada glasi:

$$h = W_0 x + \Delta W x = W_0 x + BAx$$

Matrica B inicijalizira se na nule, a matrica A nasumičnom Gaussovom inicijalizacijom, čime je promjena $\Delta W = BA$ na samom početku treniranja jednaka nuli. Model se na početku ponaša identično kao i predtrenirani model, a adaptacija se postupno uključuje kroz treniranje.

Broj parametara potrebnih za pohranu matrica A i B raste linearno s rangom r , dok originalna matrica W ima $d \times k$ parametara. Kada je r znatno manji od d i k ukupan broj novih trenabilnih parametara postaje zanemarivo malen u odnosu na broj parametara cijelog modela. LoRA se može primijeniti na bilo koji gusto povezani sloj u neuronskoj mreži, konkretno na matrice projekcije *query*, *key* i *value*, dok MLP module ostavljaju potpuno zamrznutima.

2.3. Metrike za evaluaciju modela

Neuronska mreža [3] bi uz svoju predikciju trebala pružiti kalibriranu mjeru pouzdanosti. To jest, vjerojatnost pridružena predviđenoj klasi trebala bi odgovarati stvarnoj vjerojatnosti da je predikcija točna. *Expected Calibration Error (ECE)* je metoda skalarne sažete statistike kalibracije. Ona predstavlja razliku u očekivanju između pouzdanosti i točnosti. Funkcionira tako da particionira predikcije u M jednako razmaknutih binova i izračunava ponderiranu prosječnu razliku. ECE se koristi kao primarna empirijska metrika za mjerenje kalibracije.

Procjena uspješnosti klasifikacije [4] prikazuje se matricom zabune. Ona prikazuje odnos između stvarnih oznaka primjera i oznaka koje je model predvidio. Za svaku klasu definiraju se četiri veličine: broj primjera koje je model ispravno prepoznao kao pripadnike te klase (*true positive*, TP), broj primjera koje je ispravno prepoznao kao nepripadnike (*true negative*, TN), broj primjera koje je pogrešno dodjelio toj klasi iako joj ne pripadaju (*false positive*, FP), te broj primjera koji zapravo

pripadaju klasi, ali ih model nije prepoznao (*false negative*, FN). Kod višeklasnog problema matrica zabune poprima oblik kvadratne tablice dimenzija jednakih broju klasa, pri čemu se za svaku pojedinu klasu prethodno spomenute četiri veličine računaju zasebno, promatrajući tu klasu kao pozitivnu nasuprot svim ostalima.

Standardne klasifikacijske metrike izvode se iz matrice zabune. Točnost predstavlja udio svih ispravno klasificiranih primjera u ukupnom broju primjera. Preciznost za pojedinu klasu pokazuje koliki je udio primjera koje je model proglasio tom klasom doista i pripadao toj klasi, dok odziv pokazuje koliki je udio stvarnih primjera te klase model uspio ispravno prepoznati. Ove dvije mjere se u praksi često kombiniraju u F1 metriku koja predstavlja njihov odnos i time kažnjava modele kod kojih je jedna od te dvije veličine znatno niža od druge.

Makro F1 mjera računa se tako da se preciznost i odziv prvo izračunaju zasebno za svaku klasu, zatim usrednje jednostavnom aritmetičkom sredinom preko svih klasa, nakon čega se iz tih usrednjenih vrijednosti izračuna F1 mjera. Ovakav pristup usrednjavanju daje jednaku važnost svakoj klasi neovisno o broju njezinih primjera, za razliku od F1 mikro mjere koje favorizira brojnije klase. Ova činjenica makro F1-mjeru čini prikladnijom metrikom za skupove podataka s neuravnoteženom raspodjelom klasa, poput terenskih fotografija gljiva korištenih u ovom radu.

3. SKUP PODATAKA

3.1. Izvor i sastav skupa podataka

Skup podataka [5] preuzet je sa stranice Kaggle. Sadrži slike devet vrsta gljiva. Svaka vrsta predstavlja jednu klasu. Navedene klase i broj podatkovnih primjera za svaku su Agaricus (353), Amanita (750), Boletus (1073), Cortinarius (836), Entoloma (364), Hygrocybe (316), Lactarius (1563), Russula (1148) i Suillus (311). Iz navedenog se može zaključiti kako je skup podataka neuravnotežen, to jest svaka klasa ne sadrži sličan broj podatkovnih primjera, i malen. Slike su snimljene u prirodnim uvjetima što karakterizira različit kut snimanja, različito osvjetljenje te šum. Na slici 3.1. prikazane su klase gljiva.



Slika 3.1

3.2. Podjela skupa podataka

Skup podataka podijeljen je na *train*, *test* i *validation* skupove u omjeru 70 : 15 : 15. Korištena je stratifikacija, koja dijeli skup podataka tako da svaki novi skup ima isti omjer klasa kao i originalni. Pošto skup podataka nije ravnomjeran, nužno je njeno korištenje kako se model nebi naučio većinom na istim klasama ili kako neke klase uopće nebi bile u testnim i validacijskim podacima. Pri nasumičnom dijeljenju na skupove *seed* je fiksiran što osigurava da svaki put iste slike završe u istom skupu. Na slici 3.2. prikazan je programski kod podjele skupa podataka.

```
splitfolders.ratio(  
    str(DATASET_DIR),  
    output = str(SPLIT_DATASET_DIR),  
    seed = 42,  
    ratio = (0.7, 0.15, 0.15)  
)
```

Slika 3.2

3.3. Predobrada slika i augmentacije podataka

Slike su promijenjene na veličinu 224 x 224 piksela jer transformer očekuje takav ulaz. Augmentacije trening slika umjetno povećavaju raznolikost skupa podataka. U ovom radu korištene augmentacije su nasumično zrcaljenje po horizontalnoj osi i zakretanje za petnaest stupnjeva. Model tijekom treniranja vidi slike u različitim položajima. Također, nasumično se mijenja kontrast i svjetlina slike što čini model manje osjetljivim na razlike u osvjetljenju i kontrastu na fotografijama. Slike iz svih skupova se normaliziraju tako da su skalirane kao slike iz ImageNet skupa jer je model predtreniran na tom skupu. Sve slike pretvaraju se u tenzore. Na slici 3.3. prikazan je programski kod predobrade i augmentacije podataka.

```

train_transforms = T.Compose([
    T.Resize((224,224)),
    T.RandomHorizontalFlip(),
    T.RandomRotation(15),
    T.ColorJitter(brightness = 0.2, contrast = 0.2),
    T.ToTensor(),
    T.Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225])
])

eval_and_test_transforms = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.Normalize(mean = [0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

```

Slika 3.3

4. METODOLOGIJA

4.1. Predtrenirani model

Za predtrenirani model uzet je vizualni transformer verzije "google/vit-base-patch16-224". Izlazni sloj prilagođen je broju klasa problema. Na slici 4.1. prikazan je programski kod inicijaliziranja vizualnog transformera.

```
num_classes = len(train_dataset.classes)
MODEL_NAME = "google/vit-base-patch16-224"

base_model_lora = AutoModelForImageClassification.from_pretrained(
    MODEL_NAME,
    num_labels = num_classes,
    ignore_mismatched_sizes = True
)
```

Slika 4.1

4.2. LoRA i puni fine tuning

Kod LoRA metode ne treniraju se svi parametri. Velika većina parametara se zamrzne. Kod punog *fine tuning*a nema zamrzavanja parametara nego se svi treniraju. Ispisom broja trenirabilnih parametara obje metode zaključuje se da je LoRA metoda jednostavnija za treniranje. Naime, ona trenira 301 833 od sveukupno 86 107 410 parametara koje trenira puni *fine tuning*. Na slici 4.2. prikazan je programski kod konfiguracije LoRA adaptera.

```
lora_config = LoraConfig(
    r = 8, #rang
    lora_alpha = 16, # 2 * r
    target_modules = ["q_proj", "v_proj"],
    lora_dropout = 0.1,
    bias = "none",
    modules_to_save = ["classifier"]
)
```

Slika 4.2

Rang adaptera parametar je koji određuje koliko će se parametara trenirati. Veći rang znači i veći broj trenirabilnih parametara. Parametar `lora_alpha` određuje koliko snažno LoRA adapter utječe na izlaz osnovnog modela. U ovom slučaju `alpha` je 16, a u odnosu na rang 8 daje faktor skaliranja 2. Za ciljne slojeve odabrani su *query* i *value*. To znači da se LoRA dodaje samo na te slojeve. Dropout od 10 % primjenjuje se na LoRA dio tijekom treniranja. On služi kao oblik regularizacije. Ideja je da se tijekom treniranja nasumično zanemari dio aktivacija kako se model ne bi previše prilagodio trening podacima. Parametar *bias* govori LoRA konfiguraciji kako postupiti s *bias* parametrima. Postavkom "bias='none'" *bias* parametri se ne uključuju u LoRA treniranje. Vrijednost parametra `modules_to_save` označava da se tijekom treniranja uz LoRA adaptore također trenira i sprema klasifikator.

4.3. Postupak treniranja

Postupak za treniranje modela je isti za oba pristupa kako bi se valjano mogli usporediti LoRA i puni *fine tuning*. Za optimizator je uzet *AdamW*. On određuje kako se mijenjaju težine neuronske mreže tijekom treniranja. *Learning rate* određuje veličinu koraka kojom model mijenja svoje parametre. Previsoka vrijednost dovodi do toga da model teže uči jer pravi prevelike korake. Premala vrijednost ovog parametra dovodi do presporog učenja modela. *Weight decay* je oblik regularizacije koji ograničava pretjerano velike vrijednosti težina i može pomoći u smanjenju overfittinga. Funkcija gubitka je *CrossEntropyLoss*. Ona govori modelu koliko je njegova predikcija loša. Model se trenira u deset epoha sa veličinom *batcha* trideset dva. To znači da model kroz cijeli skup podataka prolazi deset puta, a odjednom obrađuje skup od trideset dvije slike. Treniranje modela provedeno je korištenjem GPU-a u Google Colab okruženju. Na slici 4.3. prikazan je isječak programskog koda `train_model` metode koji prikazuje postavljanje gore navedenih parametara.

```
def train_model(model, train_loader, val_loader, epochs=10, lr=5e-5, verbose=True):  
    model = model.to(device)  
    optimizer = AdamW(filter(lambda p: p.requires_grad, model.parameters()),  
                        lr=lr, weight_decay=0.01)  
    criterion = torch.nn.CrossEntropyLoss()
```

Slika 4.3

4.4. Ablacijski eksperimenti

Provedena je ablacija ranga LoRA adaptera kako bi se testirao utjecaj ovog parametra na performanse modela. Na slici 4.4. prikazan je isječak programskog koda ablacije ranga. Izabrani rangovi su četiri, osam i šesnaest. Navedeni rangovi su izabrani proizvoljno. Za svaki model svi parametri osim ranga su isti kako bi usporedba bila valjana i kako bi se valjano testirao utjecaj ranga .

```
for r in [4, 8, 16]:
    cfg = LoraConfig(
        r=r, lora_alpha=2*r,
        target_modules=["q_proj", "v_proj"],
        lora_dropout=0.1, bias="none",
        modules_to_save=["classifier"],
    )
```

Slika 4.4

Također je provedena i ablacija ciljnih slojeva. Na slici 4.5. prikazan je isječak programskog koda ablacije ciljnih slojeva. Navedene kombinacije ciljnih slojeva su izabrane proizvoljno. Za svaki model svi parametri osim ciljnih slojeva su isti kako bi usporedba bila valjana i kako bi se valjano testirao utjecaj odabira ciljnih slojeva .

```
target_module_sets = {
    "q,v": ["q_proj", "v_proj"],
    "q,k,v": ["q_proj", "k_proj", "v_proj"],
    "q,k,v,o": ["q_proj", "k_proj", "v_proj", "o_proj"],
}

layer_results = []

for name, modules in target_module_sets.items():
    cfg = LoraConfig(
        r=8,
        lora_alpha=16,
        target_modules=modules,
        lora_dropout=0.1,
        bias="none",
        modules_to_save=["classifier"],
    )
```

Slika 4.5

4.5. Mjerenje memorije i računalne učinkovitosti

Na slici 4.6. prikazan je programski kod metode `count_trainable_params`. Funkcija zbraja pojavljivanje svih parametara kojima je vrijednost `requires_grad` `True`. Takva vrijednost ovog parametra označava da je parametar trenabilan, a ne zamrznut. Uz broj trenabilnih parametara vraća i broj ukupnih parametara.

```
def count_trainable_params(model):  
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)  
    total = sum(p.numel() for p in model.parameters())  
    return trainable, total
```

Slika 4.6

Na slici 4.7. prikazan je isječak programskog koda `train_model` funkcije koji prikazuje računanje performansi treniranja. Vrijeme trajanja treniranja računa se oduzimanjem vremena koje je bilo na početku treniranja od trenutnog vremena. Također se mjeri najveća količina GPU memorije koju je model koristio kod treniranja. Pozivom funkcije `count_trainable_params` dobiva se broj ukupnih i trenabilnih parametara.

```
train_time = time.time() - t0  
peak_mem_gb = torch.cuda.max_memory_allocated() / 1e9 if torch.cuda.is_available() else None  
trainable_params, total_params = count_trainable_params(model)
```

Slika 4.7

4.6. Evaluacijski postupak

U ovom radu modeli su evaluirani korištenjem metrika kao što su točnost, makro F1 i ECE. Također su evaluirani i prikazom matrice zabune. Na slici 4.8 prikazan je programski kod funkcije `expected_calibration_error`. Ona računa ECE, odnosno očekivanu pogrešku kalibracije modela. Funkcija prima vjerojatnosti koje je model dao za svaku klasu, stvarne oznake slika te broj intervala na koje je potrebno podijeliti raspon od 0 do 1. Za svaki razred računa se prosječna sigurnost modela i stvarna točnost te se njihova apsolutna razlika ponderira udjelom predikcija u tom intervalu. Konačni ECE je zbroj tih ponderiranih razlika. Vrijednost ECE-a bliža nuli znači da je model bolje kalibriran.

```

def expected_calibration_error(probs, labels, n_bins=10):
    confidences = probs.max(axis=1)
    predictions = probs.argmax(axis=1)
    accuracies = (predictions == labels).astype(float)

    bin_edges = np.linspace(0, 1, n_bins + 1)
    ece = 0.0
    bin_stats = []

    for i in range(n_bins):
        lo, hi = bin_edges[i], bin_edges[i+1]
        mask = (confidences > lo) & (confidences <= hi) if i > 0 else (confidences >= lo) & (confidences <= hi)
        if mask.sum() == 0:
            bin_stats.append((lo, hi, 0, np.nan, np.nan))
            continue
        bin_acc = accuracies[mask].mean()
        bin_conf = confidences[mask].mean()
        bin_weight = mask.sum() / len(confidences)
        ece += bin_weight * abs(bin_acc - bin_conf)
        bin_stats.append((lo, hi, mask.sum(), bin_acc, bin_conf))

    return ece, bin_stats

```

Slika 4.8

5. REZULTATI

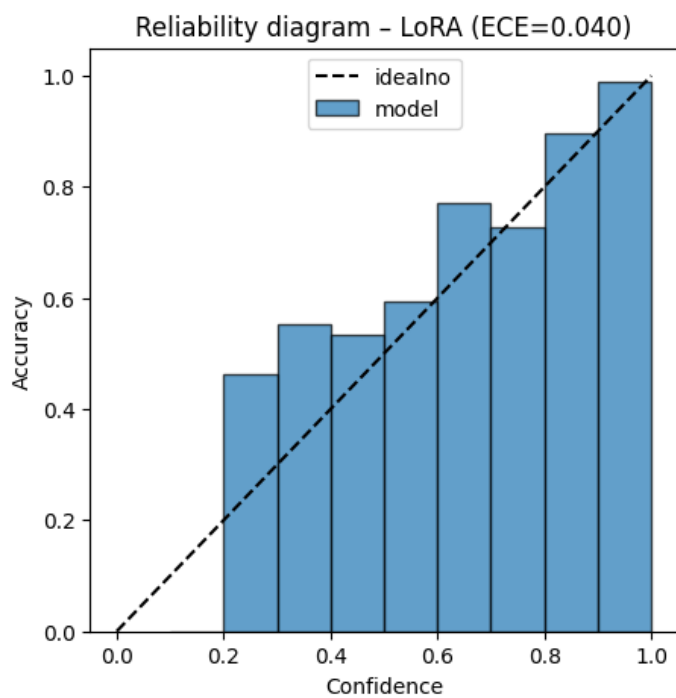
5.1. Usporedba LoRA i full fine tuninga

U tablici je prikazana usporedba LoRA i *full fine tuning metoda* prema točnosti treniranja i točnosti evaluacije nakon desete epohe. Može se primjetiti da je točnost evaluacije veća za *full fine tuning*, što dovodi do zaključka da je ova metoda u ovom slučaju bolja. Međutim prema točnosti treniranja vidi se da je *full fine tuning* dostigao overfitting. Sudeći po tome da je LoRA metoda trenirala samo 35 % parametara, iznjedrila je savršene rezultate. Potrošeni vremenski i prostorni resursi su puno manji kod LoRA metode. Ovo je čini savršenom metodom za male skupove podataka i za slučajeve gdje je cilj dobiti kvalitetan model uz uštedu resursa.

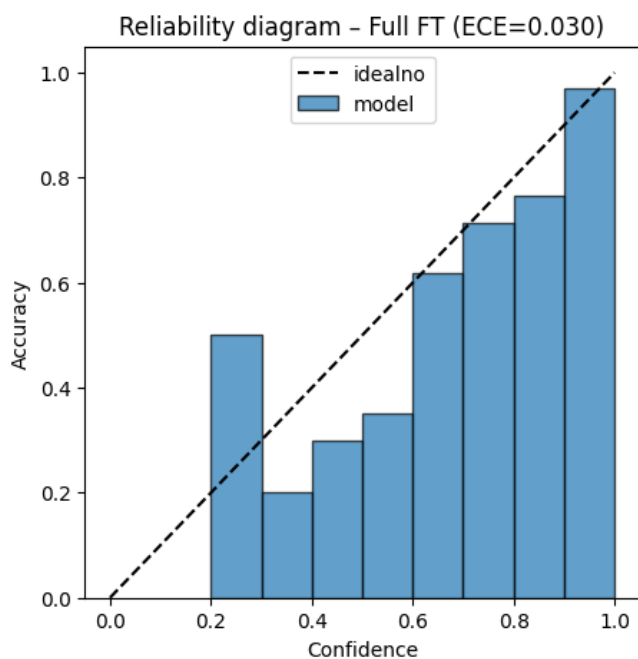
Metoda	Točnost treniranja	Točnost evaluacije	Najveća korištena količina GPU memorije (GB)	Trajanje treniranja (s)
LoRA	89.31%	87.62%	3.448	1266.15
Full fine tuning	98.85%	92.81%	5.269	1636.07

Osim točnosti, uspoređena je i kalibraciju modela pomoću ECE metrike. LoRA je ostvarila ECE od 0.0399, dok je *full fine-tuning* ostvario 0.0297. Budući da je ECE bolji što je bliži nuli, *full fine-tuning* pokazuje nešto bolju kalibraciju. To znači da je njegova procijenjena sigurnost u predikcije bolje usklađena sa stvarnom točnošću. Na slici 5.1 i 5.2 prikazani su dijagrami pouzdanosti za ove dvije metode. X os predstavlja vjerojatnost koju model dodjeljuje svojim predviđanjima, to jest sigurnost u svoju odluku. Y os prikazuje stvarnu točnost modela unutar pojedinih razreda. Crna isprekidana linija predstavlja savršenu kalibraciju za svaki razred. Kod savršene kalibracije vrijednosti na x i y osi su jednake. Plavi stupci prikazuju stvarno ponašanje modela grupirano u intervale pouzdanosti. Poželjno je da vrhovi stupaca budu što bliže isprekidanoj crnoj liniji. Na

grafovima se može primjetiti da kod LoRA modela postoji veće odstupanje stvarnog ponašanja modela od idealnih vrijednosti nego kod drugog modela. Ovo opravdava manju vrijednost ECE za *full fine tuning*.

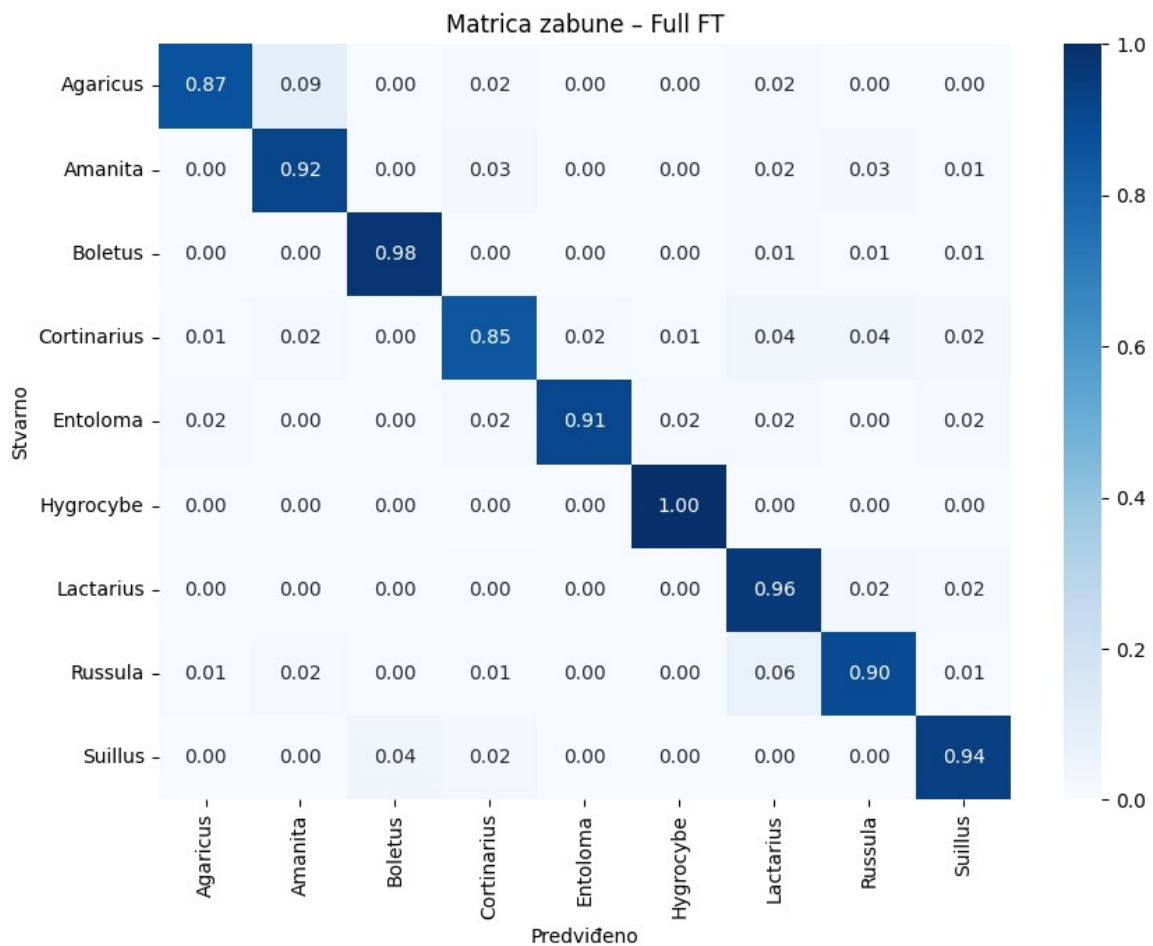


Slika 5.1

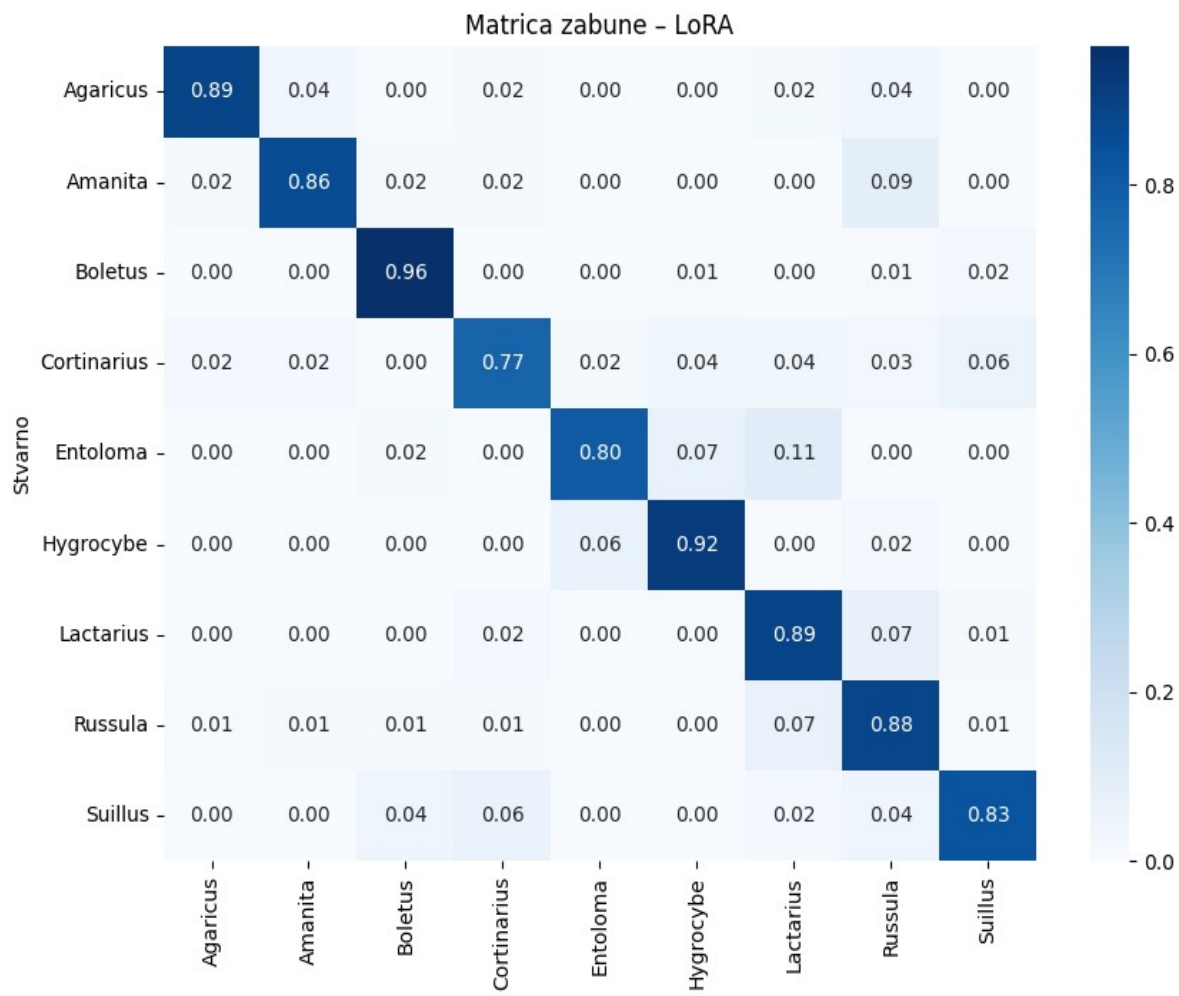


Slika 5.2

Na slikama 5.3 i 5.4 prikazane su matrice zabune za oba modela. Matrica zabune za *full fine tuning* pokazuje da je klasa Cortinarius imala najmanju točnost. Najviše krivih predviđanja bilo je kod klase Agaricus koju se ponekad predviđalo klasom Amanita. Prema matrici zabune LoRA metoda ima malo lošiji model, ali zanimljivo je da klasa Cortinarius i ovdje ima najmanju točnost.



Slika 5.3



Predvideno
Slika 5.4

5.2. Primjeri pogrešne klasifikacije

Na slici 5.5. prikazani su primjeri pogrešnih kvalifikacija kod LoRA metode.



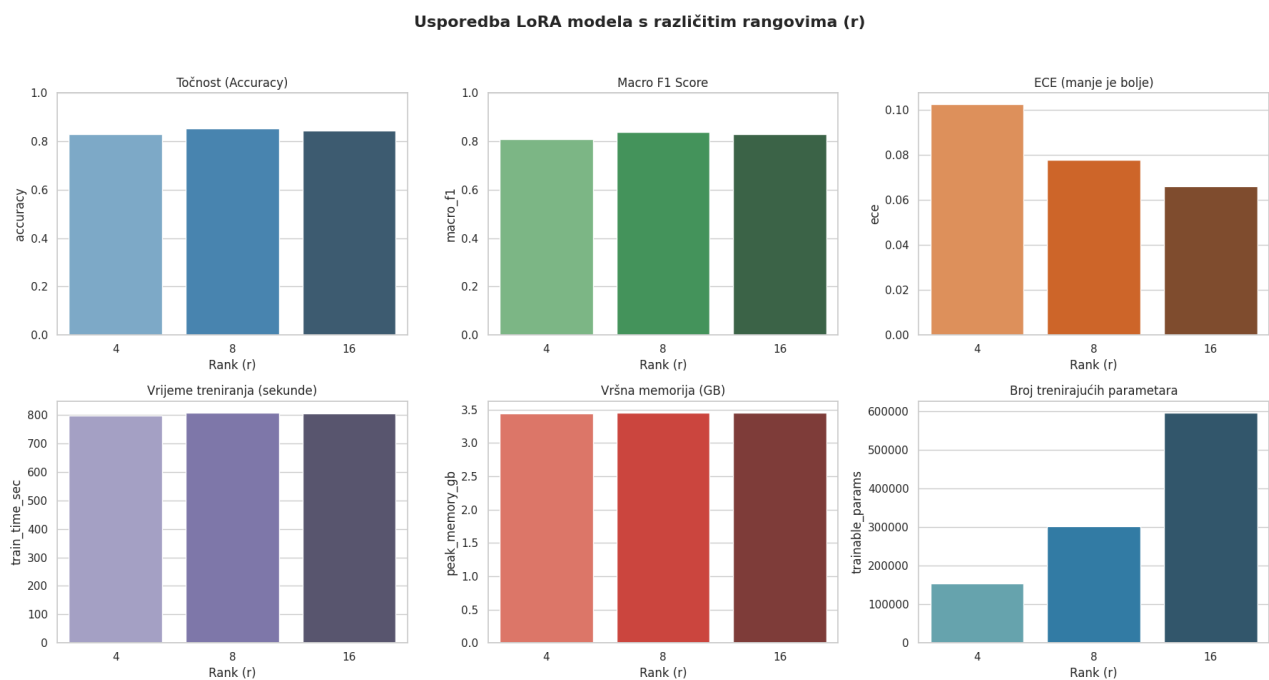
Slika 5.5

5.3. Rezultati ablacijskih eksperimenata

U tablici se može primjetiti da su korišteni vremenski i memorijski resursi slični za sva tri ranga, tako da se može zaključiti kako rang nema velik utjecaj na potrošnju resursa. Metoda sa rangom jednakim četiri je najlošija, ali metode sa rangovima osam i šesnaest imaju slične rezultate. Pošto metoda sa rangom šesnaest trenira skoro duplo više parametara, najpametnije je odabrati rang jednak osam.

Rang	Točnost	Makro F1	ECE	Trenirabilni parametri	Najveća korištena količina GPU memorije (GB)	Trajanje treniranja (s)
4	0.82	0.80	0.10	154377	3.443	798.54
8	0.85	0.83	0.07	301833	3.448	808.54
16	0.84	0.83	0.06	596745	3.457	805.55

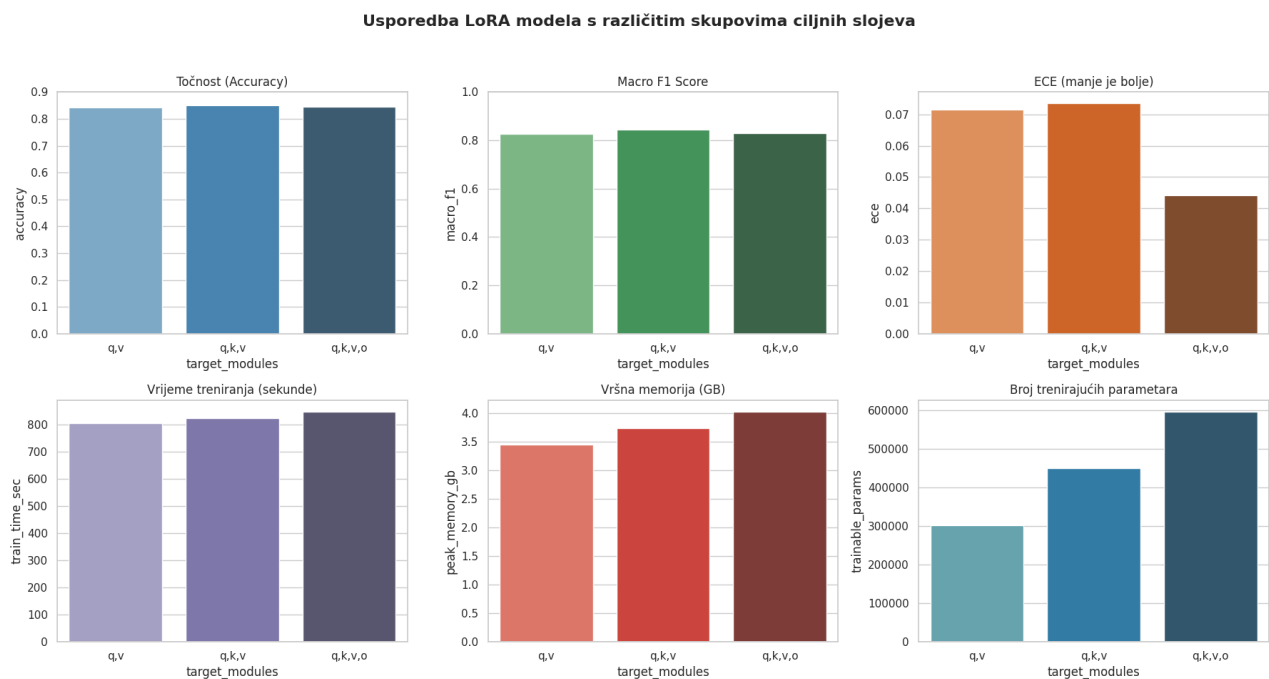
Na slici 5.6 navedeni podaci vizualizirani su grafički.



Slika 5.6

Ciljni slojevi	Točnost	Makro F1	ECE	Trenirabiln i Parametri	Najveća korištena količina GPU memorije (GB)	Trajanje treniranja (s)
q,v	0.84	0.82	0.07	301833	3.448	804.87
q,k,v	0.85	0.84	0.07	449289	3.738	822.41
q,k,v,o	0.84	0.83	0.04	596745	4.033	847.05

U tablici se može primjetiti da povećanjem broja ciljnih slojeva raste i potrošnja memorije i trajanje treniranje. Vrijednost ostalih parametara je slična osim osjetno boljeg ECE-a kod zadnjeg skupa ciljnih slojeva. Bolji ECE dolazi sa većim trošenjem resursa. Prva dva skupa slojeva donose slične rezultate. Na slici 5.7. navedeni podaci prikazani su grafički u obliku grafova.



Slika 5.7

5.4. Preporuke kod treniranja manjih skupova podataka

Rezultati eksperimenta upućuju na to da je pri treniranju vizualnih transformera na malim, neuravnoteženim skupovima podataka poput terenskih fotografija gljiva LoRA fine-tuning bolji izbor od punog fine-tuninga, budući da uz znatno manji broj trenabilnih parametara postiže konkurentnu točnost. Dok puni fine-tuning brzo dosegne visoku točnost na trening skupu uz znakove prenaučivosti na validacijskom skupu. Preporučuje se koristiti makro F1-mjeru i ECE uz točnost radi realnije procjene uspješnosti na rijetkim klasama, te odabrati umjeren rang LoRA adaptera i ciljati prvenstveno query i value slojeve, jer modeli s više ciljnih slojeva imaju slične rezultate, ali troše puno više memorije. Konačno, stratificirana podjela podataka i augmentacije prilagođene domeni dodatno pridonose stabilnijem i pouzdanijem treniranju na ograničenoj količini podataka.

6. ZAKLJUČAK

U ovom radu istreniran je vizualni transformer za klasifikaciju devet vrsta gljiva na temelju terenskih fotografija, uz primjenu LoRA adaptera kao učinkovite *fine tuning* metode. Rezultati LoRA pristupa sustavno su uspoređeni s rezultatima punog *fine-tuninga* na istom predtreniranom *backboneu*, uz identične augmentacije i identičan postupak treniranja, kako bi usporedba bila korektna. Zaključeno je da iako metoda punog *fine tuninga* ima bolje rezultate, teži overfittingu. LoRA modeli proizvode dovoljno dobre rezultate uz puno manji broj parametara za treniranje. Analiza matrica zabune i konkretnih pogrešno klasificiranih primjera otkrila je da se najveći dio pogrešaka modela koncentrira među rodovima gljiva koji dijele izraženu vizualnu sličnost. Provedene ablacijske analize ranga LoRA adaptera i odabira ciljnih slojeva pokazale su da povećanje ranga adaptera iznad određene granice ne donosi bolje performanse. Proširivanje LoRA adaptacija izvan osnovnih slojeva na dodatne slojeve nije donjelo primjetljivo povećanje rezultata, a povećalo je memorijske i vremenske zahtjeve.

Buduća istraživanja trebala bi proširiti skup podataka novim podatkovnim primjerima i novim vrstama gljiva. Također, bilo bi poželjno isprobati još neke metode osim LoRA adaptera i punog *fine tuninga*. Ablacijske eksperimente bilo bi poželjno provesti sa većim brojem epoha i većim brojem različitih eksperimenata.

LITERATURA

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, "An image is worth 16X16 words: transformers for image recognition at scale ", ICLR, lipanj 2021., dostupno na : <https://arxiv.org/pdf/2010.11929>
- [2] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, "LoRA: Low-rank adaptation of large language models", listopad 2021., dostupno na : <https://arxiv.org/pdf/2106.09685>
- [3] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, "On Calibration of Modern Neural Networks", *Proceedings of the 34th International Conference on Machine Learning*, PMLR, 2017., dostupno na : <https://proceedings.mlr.press/v70/guo17a.html>
- [4] M. Sokolova, G. Lapalme, "A systematic analysis of performance measures for classification tasks", Elsevier Ltd., 2009., dostupno na : https://www.researchgate.net/publication/222674734_A_systematic_analysis_of_performance_measures_for_classification_tasks
- [5] Mycologist's society of Northern Europe, "Mushrooms classification - Common genus's images", Kaggle, dostupno na : <https://www.kaggle.com/datasets/maysee/mushrooms-classification-common-genuss-images>

POPIS SLIKA

Slika 2.1.....	2
Slika 3.1.....	4
Slika 3.2.....	5
Slika 3.3.....	6
Slika 4.1.....	7
Slika 4.2.....	7
Slika 4.3.....	8
Slika 4.4.....	9
Slika 4.5.....	9
Slika 4.6.....	10
Slika 4.7.....	10
Slika 4.8.....	11
Slika 5.1.....	13
Slika 5.2.....	13
Slika 5.3.....	14
Slika 5.4.....	15
Slika 5.5.....	16
Slika 5.6.....	17
Slika 5.7.....	18